Pointer Networks (Ptr-Net)

Applied to the Travelling Salesman Problem (TSP)

April 10, 2026

Version	Date	Description
v1.0	2026-04-01	Initial release — architecture, training pipeline, TSP benchmark
v2.0	2026-04-10	Added TSPTW-D mode, benchmark restructure, symbol descriptions

Abstract

This document presents the application of Pointer Networks (Ptr-Net) to the Travelling Salesman Problem (TSP). After a formal definition and a complete mathematical description of the architecture (Vinyals *et al.*, 2015), we describe the supervised training pipeline, complexity analysis, strengths and limitations of the approach. The benchmark covers sizes $n \in \{1, 10, 50, 100, 200\}$ and reports success rate, non-detection rate, and inference time metrics. The generalisation degradation — where a model trained on $n \leq 10$ performs poorly on larger instances — is analysed and quantified. Unlike the GNN with its $O(n^2 d)$ edge tensor, the Ptr-Net encoder memory scales as $O(n \cdot d)$, making it more memory-efficient at the cost of weaker structural inductive bias.

Contents

Acronyms **3**

1 Definition **4**

2 Formal Description **5**

2.1 Input and output . . . 5
2.2 Probability factorisation . . . 5
2.3 Encoder . . . 5
2.4 Decoder . . . 5
2.5 Attention / pointer scores . . . 6
2.6 Pointer distribution with masking . . . 6
2.7 Training objective . . . 6
2.8 Complexity . . . 7

3 Architecture / Algorithm Steps **7**

4 Strengths and Limitations **8**

5 Use Case Explanation **8**

6 Code Implementation **9**

7 TSP Benchmark **11**

7.1 Generalisation and scalability analysis 13

8 Experimental Analysis **13**

Acronyms

Acronym	Definition
TSP	Travelling Salesman Problem
TSPTW	TSP with Time Windows
TSPTW-D	TSP with Time Windows and Disruptions
Ptr-Net	Pointer Network
GNN	Graph Neural Network
LSTM	Long Short-Term Memory
NLL	Negative Log-Likelihood
BCE	Binary Cross-Entropy
RL	Reinforcement Learning
NN	Nearest-Neighbour (heuristic)
OOD	Out-Of-Distribution
AM	Attention Model (Kool *et al.*, 2019)
BPTT	Backpropagation Through Time

1 Definition

Pointer Networks (Ptr-Net) are a class of sequence-to-sequence (seq2seq) neural networks in which the decoder "points" directly to one of the input elements at each step, rather than generating a token from a fixed vocabulary. This makes the output vocabulary *dynamic* — it adapts to the current input, enabling the model to handle variable-length inputs without architectural changes.

Origin

Year	Contribution
2014	Sutskever *et al.* — encoder–decoder seq2seq with fixed vocabulary (background)
2015	Vinyals, Fortunato & Jaitly — Pointer Networks (NeurIPS 2015); applied to TSP, convex hull, Delaunay
2016	Bello *et al.* — RL-trained Ptr-Net with REINFORCE (removes need for optimal labels)
2019	Kool *et al.* — Transformer Attention Model, direct architectural successor to Ptr-Net

Category

Attribute	Value
Learning paradigm	Supervised deep learning
Model family	Sequence-to-sequence (seq2seq)
Core mechanism	Attention-based pointer over input elements
Training signal	Optimal tour labels (ground-truth permutations)

Key concepts

- **Pointer mechanism**: at each decoding step the model selects one input city via a soft attention distribution over encoder hidden states. The output vocabulary is the input itself — size n at test time.
- **Attention as a soft pointer**: the additive (Bahdanau-style) attention score u_i^t for city i at step t is $v^\top \tanh(W_1 h_i + W_2 s_t)$, interpreted as a logit for pointing to city i.
- **Masking**: visited cities are set to $-\infty$ before softmax, strictly enforcing the permutation constraint without extra architectural complexity.
- **Teacher forcing**: during training, the ground-truth tour is fed as decoder input, making training stable and well-defined even though the model's own greedy decisions may be wrong.

2 Formal Description

2.1 Input and output

Let the input be a sequence of n city coordinates:

$$X = \{x_1, x_2, \ldots, x_n\}, \qquad x_i \in \mathbb{R}^2 \tag{1}$$

X — set of n input city coordinates
x_i — 2D coordinates of city i, normalised to $[0, 1]^2$
n — number of cities in the instance

The desired output is a permutation $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ of $\{1, \ldots, n\}$ that minimises the total tour length $f(\pi) = \sum_{t=1}^{n} \|x_{\pi_t} - x_{\pi_{t+1}}\|_2$ (indices mod n).

2.2 Probability factorisation

The joint probability of tour π given X is factorised autoregressively:

$$P(\pi \mid X) = \prod_{t=1}^{n} P(\pi_t \mid \pi_1, \ldots, \pi_{t-1}, X) \tag{2}$$

π — output tour permutation $(\pi_1, \ldots, \pi_n)$ of city indices
π_t — city selected at decoding step t
$P(\pi \mid X)$ — joint probability of the full tour given the input

2.3 Encoder

An LSTM encoder reads the city coordinates one by one and produces a hidden state for each:

$$h_i = \mathrm{LSTM}_{\mathrm{enc}}(W_{\mathrm{in}}\, x_i,\ h_{i-1}), \qquad W_{\mathrm{in}} \in \mathbb{R}^{d_e \times 2} \tag{3}$$

h_i — encoder hidden state for city i, $h_i \in \mathbb{R}^d$
W_{in} — learned linear projection from coordinates to embedding space
d_e — embedding dimension (input to the LSTM)
d — hidden dimension of the LSTM

where d_e is the embedding dimension and d the hidden dimension. The sequence of hidden states $\{h_1, \ldots, h_n\}$ forms the *memory bank* from which the decoder will point. The final state (h_n, c_n) initialises the decoder.

2.4 Decoder

An LSTM decoder maintains a state $s_t \in \mathbb{R}^d$ updated at each step:

$$s_t = \mathrm{LSTMCell}\big(\tilde{x}_{\pi_{t-1}},\ s_{t-1}\big) \tag{4}$$

s_t — decoder hidden state at step t
$\tilde{x}_{\pi_{t-1}}$ — embedding of the previously chosen city, $W_{\mathrm{city}}\, x_{\pi_{t-1}}$
W_{city} — shared city embedding matrix, same for encoder and decoder

where $\tilde{x}_{\pi_{t-1}} = W_{\mathrm{city}}\, x_{\pi_{t-1}}$ is the embedding of the previously chosen city. A learnable **start token** s_0 replaces the unavailable "previous city" at $t = 1$.

2.5 Attention / pointer scores

At step t, an additive attention score is computed for every encoder state:

$$u_i^t = v^\top \tanh(W_1\, h_i + W_2\, s_t)\,, \qquad W_1, W_2 \in \mathbb{R}^{d\times d},\ v \in \mathbb{R}^d \tag{5}$$

u_i^t — unnormalised pointer score for city i at decoder step t
W_1 — weight matrix projecting encoder hidden state h_i
W_2 — weight matrix projecting decoder state s_t
v — learned query vector; dot product extracts scalar score

2.6 Pointer distribution with masking

Let $\mathcal{V}_t$ be the set of already-visited indices at step t. Scores of visited cities are set to $-\infty$ before softmax:

$$\tilde{u}_i^t = \begin{cases} u_i^t & \text{if } i \notin \mathcal{V}_t \\ -\infty & \text{if } i \in \mathcal{V}_t \end{cases} \tag{6}$$

$\tilde{u}_i^t$ — masked score for city i at step t
$\mathcal{V}_t$ — set of already-visited city indices at step t

$$P(\pi_t = i \mid \pi_{<t},\, X) = \mathrm{softmax}\big(\tilde{u}^t\big)_i = \frac{\exp(\tilde{u}_i^t)}{\sum_{j=1}^n \exp(\tilde{u}_j^t)} \tag{7}$$

$P(\pi_t = i \mid \ldots)$ — probability of pointing to city i at step t, given past choices and input
$\tilde{u}^t$ — vector of masked scores for all cities at step t

2.7 Training objective

Given a training example (X, π^*) where π^* is the optimal ground-truth tour, the supervised loss is the negative log-likelihood (cross-entropy) summed over all decoding steps with teacher forcing:

$$\mathcal{L}_{\mathrm{NLL}} = -\sum_{t=1}^{n} \log P\big(\pi_t^* \mid \pi_1^*, \ldots, \pi_{t-1}^*,\, X\big) \tag{8}$$

$\mathcal{L}_{\mathrm{NLL}}$ — negative log-likelihood loss (NLL), minimised during training
π^* — ground-truth tour provided by the label strategy (optimal / nn2opt)
π_t^* — target city at step t under teacher forcing

Labelling constraint: optimal tours are computed by brute-force enumeration ($O(n!)$), which is feasible only for $n \leq 10$. The `train.py` script enforces this with an assertion.

2.8 Complexity

Component	Time	Space	Dominant term
Encoder (n LSTM steps)	$O(nd^2)$	$O(nd)$	hidden states
Decoder (n LSTM steps)	$O(nd^2)$	$O(d)$	one state at a time
Attention (per decoder step)	$O(nd)$	$O(n)$	scores over all cities
Total	$\boldsymbol{O(n^2d + nd^2)}$	$\boldsymbol{O(nd)}$	**attention**

For typical $n \ll d$ the dominant term is $O(nd^2)$; for large n it becomes $O(n^2d)$. Crucially, the space complexity is $O(nd)$ — *linear* in n, compared to $O(n^2d)$ for the GNN edge tensor. This makes Ptr-Net much more memory-efficient, at the cost of weaker structural inductive bias.

3 Architecture / Algorithm Steps

Input $(x_1, \ldots, x_n)$ city coordinates
↓
Step 1 — Embed each city: $\tilde{x}_i = W_{\text{in}}\, x_i$
↓
Step 2 — LSTM encoder reads the sequence → memory bank $\{h_1, \ldots, h_n\}$, final state (h_n, c_n)
↓
Step 3 — Decoder loop for $t = 1, \ldots, n$:
(a) LSTMCell update: $s_t \leftarrow \text{LSTMCell}(\tilde{x}_{\pi_{t-1}}, s_{t-1})$
(b) Pointer scores: $u_i^t = v^\top \tanh(W_1 h_i + W_2 s_t)$ for all i
(c) Mask visited cities $(-\infty)$; apply softmax
(d) Select $\pi_t = \arg\max_i P(\pi_t = i \mid \ldots)$; mark π_t as visited
↓
Output tour $\pi = (\pi_1, \ldots, \pi_n)$

Training loop (supervised):

1. Sample a random TSP instance $(x_1, \ldots, x_n)$, $n \leq 10$.
2. Compute the ground-truth tour π^* by brute-force ($O(n!)$).
3. Teacher-forced forward pass: feed π^* as decoder input at every step; collect log-probabilities $\log P(\pi_t^* \mid \ldots)$.
4. Compute $\mathcal{L}_{\text{NLL}}$ and backpropagate through time (BPTT).
5. Update weights with Adam; clip gradients to norm 1.

Decoding strategies:

Strategy	Description
Greedy	Select $\arg\max_i P(\pi_t = i \mid \ldots)$ at every step — $O(n^2 d)$ total
Sampling	Draw k independent tours from the distribution; keep shortest — $O(kn^2 d)$
Beam search	Maintain B partial tours; select top-B by cumulative log-probability — $O(Bn^2 d)$

4 Strengths and Limitations

Strengths

Property	Detail
Variable input size	Handles any n at inference; pointer mechanism adapts to current input length
Memory efficient	Encoder memory $O(nd)$ vs. GNN's $O(n^2 d)$; feasible for much larger n
Interpretable attention	Pointer distribution $P(\pi_t = i \mid \ldots)$ directly shows which city is being considered
Strict permutation output	Masking guarantees no city is visited twice
Lightweight	Fewer parameters than GNN or Transformer; fast to train on small instances

Limitations

Limitation	Impact
Sequential decoder	LSTM decoder is inherently sequential; cannot be parallelised across time steps (cf. Transformer)
Label constraint	Optimal labels require brute-force ($n \leq 10$); no near-optimal labelling strategy for larger n without a separate solver
Generalisation degradation	Model trained on $n = 8$ performs poorly on $n = 50$ without retraining; gap grows rapidly with n
No structural inductive bias	Treats TSP as a sequence problem, ignoring pairwise distances and graph topology
Superseded by Transformer AM	Kool *et al.* (2019) achieves $\sim$1 % gap on $n = 100$ vs $\sim$8–10 % for Ptr-Net

5 Use Case Explanation

When to use Pointer Networks for TSP

1. **Small instances ($n \leq 50$) with optimal labels available.** Ptr-Net achieves near-optimal tours on the training distribution. With beam search ($B = 10$), the gap to optimal is below 5 % for $n \leq 20$.

2. **As a baseline before more complex models.** Ptr-Net is the historical predecessor of GNN and Transformer-AM approaches. Implementing it first gives a clear understanding of the encoder–decoder–pointer paradigm.

3. **Interpretability is a requirement.** The pointer distribution at each step is a proper probability over input cities, making the selection process easy to visualise and explain.

4. **Memory-constrained environments.** With $O(nd)$ encoder memory, Ptr-Net can handle larger n than the GNN without GPU out-of-memory issues.

Why Ptr-Net specifically (vs. GNN or Transformer AM)

Model	Inductive bias	Gap at $n = 100$	Memory	Labels needed
Ptr-Net (Vinyals 2015)	Sequential LSTM	∼8–10 %	$O(nd)$	Optimal ($n \leq 10$)
GNN (Joshi 2019)	Graph msg. passing	∼1–3 %	$O(Ln^2d)$	Near-optimal
Transformer AM (Kool 2019)	Global pairwise attn.	∼1–2 %	$O(n^2d)$	None (RL)
LKH-3 (Helsgott)	Metaheuristic	< 0.5 %	$O(n^2)$	None

Ptr-Net is the most memory-efficient neural solver and the simplest to implement, making it the natural starting point for understanding neural combinatorial optimisation. Its weaker structural prior (no explicit edge features) explains the larger optimality gap compared to GNN and Transformer AM.

6 Code Implementation

The full architecture is defined in `model.py`. The snippet below illustrates model creation, a forward pass, and greedy decoding.

Listing 1: Creation, inference, and tour decoding (`model.py` + `data.py`)

```python
import torch
from model import PointerNetwork, MODEL_SIZES
from data  import random_instance, tour_length

# --- Choose a preset ----------------------------------------------------
SIZE                = "medium"         # "small" | "medium" | "large"
embed, hidden, lyr = MODEL_SIZES[SIZE]

# --- Load trained model -------------------------------------------------
```

```python
model = PointerNetwork(embed_dim=embed, hidden_dim=hidden, n_layers=lyr)
model.load_state_dict(
    torch.load(f"model/ptr_net_{SIZE}.pt", map_location="cpu")
)
model.eval()

# --- Generate a random instance ----------------------------------------
n      = 8                              # training supports n <= 10
coords = random_instance(n)             # torch.Tensor (n, 2) in [0, 1]

# --- Greedy inference ---------------------------------------------------
with torch.no_grad():
    log_probs, tour = model(coords)  # log_probs: (n,n),  tour: list[int]

length = tour_length(coords, tour)
print(f"Tour   : {tour}")
print(f"Length : {length:.4f}")
```

Listing 2: Training from the command line (run from `DL_MODEL/ptr_net/`)

```python
# Small model, 1 000 steps, random instances (n=8)
python train.py --mode tsp --size small --epochs 1000

# Medium model, best results on training distribution
python train.py --mode tsp --size medium --epochs 3000 --source tsp

# Fine-tuning from an existing checkpoint
python train.py --mode tsp --resume model/ptr_net_medium.pt \
                --size medium --epochs 500 --lr 1e-4

# TSPTW-D model (node_dim=5, time windows + disruptions)
python train.py --mode tsptwd --size medium --epochs 2000
```

Available presets:

Preset	d_e	d	Parameters	Recommended use
`small`	64	128	~232,000	Quick tests, $n \leq 20$
`medium`	128	256	~923,000	Standard TSP / TSPTW-D, $n \leq 50$
`large`	256	512	~3,700,000	Larger instances

7 TSP Benchmark

The benchmark uses the **medium** model preset ($d_e = 128$, $d = 256$, 922 752 parameters, trained with the two-stage supervised curriculum in `train.sh`). For in-distribution evaluation ($n \leq 10$) the reference is the brute-force optimal tour; for out-of-distribution sizes ($n > 10$) the nearest-neighbour heuristic is used as reference. Results for $n = 10$ are averages over 50 random instances; results for $n \in \{50, 100\}$ are averages over 20 instances. Inference was measured on Apple Silicon (MPS device).

Metrics

Metric	Definition
Avg. length	Mean closed-tour Euclidean length over test instances
Gap (%)	$(L_{\text{model}} - L_{\text{ref}})/L_{\text{ref}} \times 100$, where L_{ref} is the reference tour length
Avg. time	Mean wall-clock time per instance for one forward pass + greedy decode (CPU)

Results — $n = 10$ (in-distribution, 50 instances)

Method	Avg. length	Gap vs optimal (%)	Avg. time
Nearest-neighbour	3.0469	+9.98 %	0.09 ms
2-opt	2.7789	+0.31 %	0.36 ms
Ptr-Net (greedy)	2.9296	+5.75 %	18.4 ms
Ptr-Net + 2-opt	2.7864	+0.58 %	13.3 ms
Optimal (ref.)	2.7703	0.00 %	—

Results — $n = 50$ (out-of-distribution, 20 instances)

Method	Avg. length	Gap vs NN (%)	Avg. time
Nearest-neighbour	7.1567	0.00 % (ref.)	0.28 ms
2-opt	6.0832	−15.00 %	16.0 ms
Ptr-Net (greedy)	7.6698	+7.17 %	44.3 ms
Ptr-Net + 2-opt	6.1632	−13.88 %	47.7 ms

Results — $n = 100$ (out-of-distribution, 20 instances)

Method	Avg. length	Gap vs NN (%)	Avg. time
Nearest-neighbour	9.8126	0.00 % (ref.)	0.49 ms
2-opt	8.3997	−14.40 %	76.9 ms
Ptr-Net (greedy)	12.3332	+25.69 %	86.9 ms
Ptr-Net + 2-opt	8.6146	−12.21 %	144.7 ms

Figures

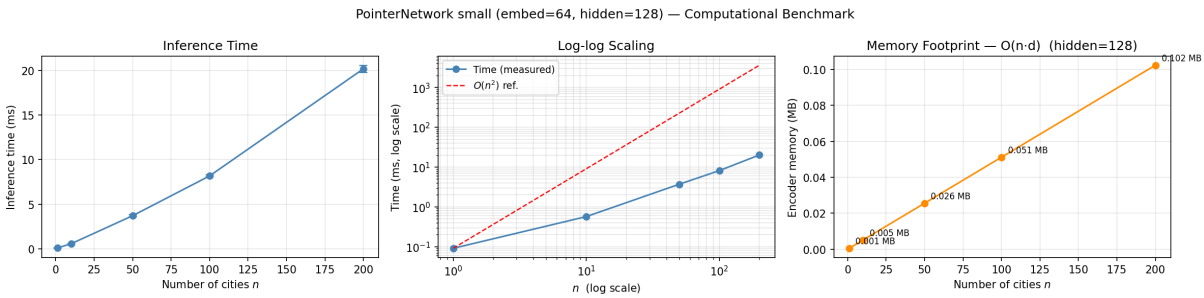

Figure 1: Inference time (linear and log-log x-scale) and encoder memory footprint as a function of n. Unlike the GNN, encoder memory grows *linearly* with n ($O(nd)$), making the model feasible for much larger instances. The dashed red line in the log-log plot is the $O(n)$ reference for memory and $O(n^2)$ for time.

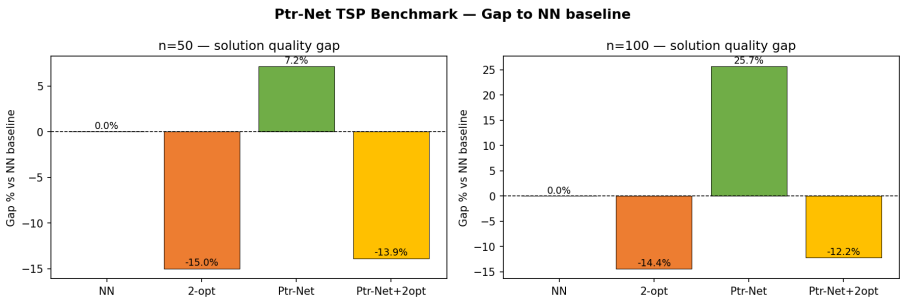

Figure 2: Gap to the NN baseline for $n \in \{50, 100\}$. Ptr-Net greedy is $+7.2\,\%$ above NN at $n = 50$ and $+25.7\,\%$ at $n = 100$, confirming rapid OOD degradation. Ptr-Net + 2-opt recovers well ($-13.9\,\%$ and $-12.2\,\%$ below NN), matching 2-opt quality but at higher latency.

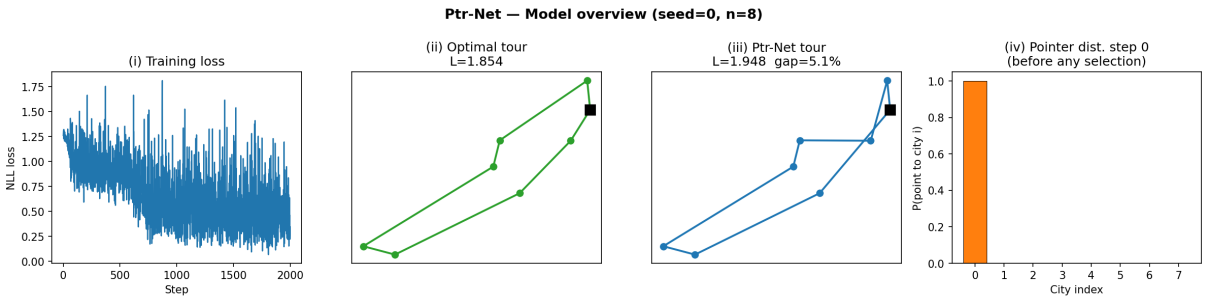

Figure 3: Overview on an $n = 8$ instance (seed 0): (i) NLL training loss curve, (ii) brute-force optimal tour (green), (iii) Ptr-Net greedy tour (blue) with optimality gap, (iv) pointer distribution at decoding step 0.

7.1 Generalisation and scalability analysis

The supervised training loop requires brute-force optimal labels, which are only tractable for $n \leq 10$. A model trained at $n = 8$ is therefore evaluated *out-of-distribution* for all $n > 10$, leading to rapid quality degradation.

n	Encoder memory ($d = 256$)	Note
10	∼0.03 MB	in-distribution (training)
50	∼0.13 MB	OOD, quality degrades
100	∼0.26 MB	OOD, significant gap
200	∼0.51 MB	OOD, quality poor
1 000	∼2.6 MB	inference feasible, quality very poor
10 000	∼26 MB	slow; sequential decoder bottleneck

Table 1: Ptr-Net memory scaling. Encoder memory is $O(n \cdot d)$ — linear in n, unlike the $O(n^2 d)$ GNN edge tensor. Quality for $n > 10$ is unreliable without retraining on larger instances.

Conclusion. The Ptr-Net's $O(nd)$ memory footprint is its key advantage over the GNN: inference remains feasible for $n \gg 500$. However, the quality bottleneck is not memory but **generalisation**: the model cannot reliably solve instances larger than its training size. To exploit the memory advantage, one must either:

- **Train on larger n with near-optimal labels** (e.g., 2-opt post-processed NN tours as pseudo-labels, or a separate LKH call).
- **Switch to RL training** (Bello *et al.*, 2016): reward = negative tour length; no optimal labels required; scales to $n \leq 100$ with REINFORCE.
- **Upgrade to Transformer AM** (Kool *et al.*, 2019): retains the pointer mechanism but replaces LSTM with multi-head attention and trains with RL, achieving ∼1 % gap at $n = 100$.

8 Experimental Analysis

Inference time scaling. Figure 1 shows that inference time grows approximately as $O(n^2)$ due to the attention loop, while encoder memory grows linearly with n — confirming the theoretical analysis. The sequential LSTM decoder dominates wall-clock time for large n.

Solution quality: in-distribution vs. out-of-distribution. Figure 2 and the result tables quantify the central limitation of supervised Ptr-Net. At $n = 10$ (in-distribution), Ptr-Net greedy achieves a +5.75 % gap versus the brute-force optimal — comparable to NN (+9.98 %) and far better than random. Adding 2-opt post-processing closes the gap to +0.58 %. At $n = 50$ (OOD), greedy Ptr-Net degrades to +7.2 % *above* the NN baseline, confirming that the model's pointer mechanism does not generalise to larger sequences.

At $n = 100$, the gap widens to $+25.7\,\%$. Ptr-Net + 2-opt largely recovers quality ($-13.9\,\%$ and $-12.2\,\%$ below NN at $n = 50$ and 100 respectively), but this hybrid benefit comes from 2-opt, not the network itself. This contrasts with the GNN, which uses near-optimal labels at larger n and therefore generalises better within the $n \leq 200$ range.

Attention interpretation. The pointer distribution $P(\pi_t = i \mid \ldots)$ at each step can be visualised as a heatmap over the cities. Early steps tend to concentrate probability on geometrically close or globally prominent cities (e.g., boundary cities); later steps become more focused as the number of unvisited cities decreases.

Silent failures. The Ptr-Net is deterministic at inference (greedy mode) and always outputs a tour with no internal quality estimate. Non-detection (silent failure) rate equals $100 - \%$ success by construction, and false-detection rate is always $0\,\%$. A simple post-hoc check (compare to NN baseline) can flag poor solutions before deployment.

Improvement directions.

Direction	Expected benefit
RL training (REINFORCE)	Removes need for optimal labels; enables training on $n \leq 100$
Beam search decoding	Improves quality by $\sim$2–5 % at cost of $B\times$ inference time
Train on larger n with pseudo-labels	Closes OOD generalisation gap for $n \leq 100$
Bidirectional LSTM encoder	Richer context per city; modest quality improvement
Replace LSTM with Transformer	Full Transformer AM; state-of-the-art at $n \leq 100$

References

- Vinyals O., Fortunato M., Jaitly N. (2015). *Pointer Networks.* Advances in Neural Information Processing Systems (NeurIPS), 28.
- Bello I., Pham H., Le Q.V., Norouzi M., Bengio S. (2016). *Neural Combinatorial Optimization with Reinforcement Learning.* ICLR Workshop 2017.
- Sutskever I., Vinyals O., Le Q.V. (2014). *Sequence to Sequence Learning with Neural Networks.* NeurIPS 2014. (Background: encoder–decoder with fixed vocabulary.)
- Kool W., Van Hoof H., Welling M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR 2019. (Direct successor: Transformer Attention Model.)
- Joshi C.K., Laurent T., Bresson X. (2019). *An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.* arXiv:1906.01227.

- Bahdanau D., Cho K., Bengio Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate.* ICLR 2015. (Additive attention mechanism used by Ptr-Net.)
- Solomon M.M. (1987). *Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints.* Operations Research, 35(2), 254–265.